STAFF REVIEW · H446 013 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. The item at index 2 — the third item.

2. A counter, condition and update.

3. The number of rows.

4. 9.

I Do — source worked steps

1. Create a working output — Run these instructions in order. The result is a 3×3 numbered board. — CODE:
Console.WriteLine("1 | 2 | 3");
Console.WriteLine("--+---+--");
Console.WriteLine("4 | 5 | 6");
Console.WriteLine("--+---+--");
Console.WriteLine("7 | 8 | 9");

2. Change one square — Change the centre of the printed board to X. This shows the limitation of storing positions inside string literals. — CODE:
Console.WriteLine("4 | X | 6");

3. Move the values into a 2D array — CODE:
string[,] board = {
{ "1", "2", "3" },
{ "4", "5", "6" },
{ "7", "8", "9" }
}; — Square 5 is board[1,1] , not board[5] , because our board uses zero-based row and column indexes.

4. Print the same values without a loop — CODE:
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]); — This code works. But how many times do the row-printing instructions repeat? What changes between lines? We'll tackle that in Lesson 014.

We Do — guided CHECK

1. Why it works
Use one Console.WriteLine() for each row.
CODE:
Console.WriteLine("1 | 2 | 3");
Console.WriteLine("4 | 5 | 6");

Console.WriteLine("7 | 8 | 9");

2. Write or explain
With zero-based indexes, what is the row/column index of the centre square (5)?
Why it works
The centre square is row 1, column 1.
CODE:
board[1,1]

3. Write or explain
After declaring a 3×3 string array, write one assignment that places X in the centre.
Why it works
CODE:
board[1,1] = "X";
No other cell changes.

4. Write or explain
Look at three separate row-print statements. What is identical, and what changes each row?
Why it works
Each statement prints three column values joined by separators. Only the row index changes: 0 → 1 → 2. This is repetition worth simplifying next lesson.

You Do — source mark schemes

1. 1. Display the board with sequence
Each Console.WriteLine() runs in order. You can produce the whole board without a loop.
CODE:
Console.WriteLine("1 | 2 | 3");
Console.WriteLine("--+---+--");
Console.WriteLine("4 | 5 | 6");
Console.WriteLine("--+---+--");
Console.WriteLine("7 | 8 | 9");

2. 2. Change the middle value
Change the third WriteLine statement to show X rather than 5.
CODE:
Console.WriteLine("4 | X | 6");
Works, but makes moves difficult to maintain because the board is stored as printed text.

3. 3. Store the nine values in a two-dimensional array
Declare the board as string[,] and change its middle cell using row 1 and column 1.
CODE:
string[,] board = {
{ "1", "2", "3" },
{ "4", "5", "6" },
{ "7", "8", "9" }
};
board[1,1] = "X";
Indexes start at zero.

4. 4. Output data from the array without loops
The three statements still follow sequence, but they now display the array values. They work after a move changes an array element.
CODE:

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

Notice: the only changing part of these statements is the row index. This is why iteration is useful next lesson.

Exit Ticket — source answer

1. False. You can display a 3×3 board with explicit statements; a loop is useful but not required.